DigiCommLab

Professor Q&A & Technical Viva Preparation Guide

Interactive Digital Communication Systems Testbed

ACADEMIC & TECHNICAL DOSSIER

Course:	**Digital Communication Systems (DCS)**
Project Domain:	Physical Layer Digital & Analog Signal Simulation Testbed
Purpose:	Comprehensive Viva Voce, Technical Evaluation, and Code Defense
Architecture:	Single-Page Client-Side Application (Next.js 16, React 19, TypeScript)
Core Modules:	`1. Sampling & Quantization` `2. Digital Coding` `3. Line Coding` `4. Modulation`

OVERVIEW

Table of Contents

Structural index of physical-layer modules, mathematical validations, and defense topics.

Section 1: Project Architecture	— Client-side execution, Float64 arrays, plotting continuity	**Q1 – Q2**
Section 2: Sampling & Quantization	— Nyquist criterion, aliasing folding, sinc interpolation	**Q3 – Q5**
Section 3: Digital Coding	— PCM, DPCM prediction loop, DM slope overload, ADM adaptation	**Q6 – Q9**
Section 4: Line Coding	— NRZ, RZ, Manchester, Bipolar AMI, Differential, and Analytical PSD	**Q10 – Q12**
Section 5: Modulation	— AM/FM, M-QAM, CPFSK/MSK, Box-Muller AWGN, and DFT spectrum	**Q13 – Q17**
Section 6: Validation and Implementation Limitations	— Benchmark tests, edge cases, and honest challenge answers	**Q18 – Q20 + Challenges**
Section 7: Code-to-Theory Mapping	— Full matrix linking features to algorithms, files, and formulas	**Master Table**
Section 8: Final Revision Sheet	— 15 core formulas, 5 key concepts, and 5 presentation pitfalls	**Summary Sheet**
Section 9: 60-Second Project Explanation	— Concise elevator pitch to begin your viva defense	**Viva Opener**

SECTION 1

Project Architecture & Execution Model

Client-side signal generation, oversampled discrete timelines, and visual continuity algorithms.

Q1. How is DigiCommLab architected, and how does the browser calculate and plot these waveforms in real time?

SIMPLE ANSWER (PRESENTATION / VIVA READY)

DigiCommLab is built with Next.js 16 and React 19, separating physical layer simulation logic from user interface rendering. The simulation algorithms are written in pure TypeScript using typed Float64 numeric arrays for fast signal processing. Whenever a slider or parameter changes, React recalculates the entire mathematical pipeline in client memory via `useMemo` hooks. The resulting points are fed into Recharts, which renders SVG paths with zero server-side latency.

TECHNICAL EXPLANATION (CODE & ARCHITECTURE)

All signal engines live in `src/utils/simulations/`. Signals are computed synchronously in browser memory. Continuous-time waveforms are simulated by choosing an oversampled discrete simulation clock $f_{\text{sim}} = \max(10000, 50f_m)$ to prevent plotting artifacts. To keep the UI responsive at 60 FPS, high-resolution arrays (often 2,500–5,000 floats) are downsampled to 400–600 display points before binding to Recharts SVG components, while preserving exact transition points and sample coordinates.

GOVERNING FORMULA

Simulation timeline discretization:

$$t[i] = \frac{i}{f_{\text{sim}}}, \quad i \in \{0, 1, \dots, N-1\}, \quad N = \lfloor T_{\text{duration}} \cdot f_{\text{sim}} \rfloor$$

where T_{duration} is the simulated time window (50 ms to 200 ms).

HOW IT APPEARS IN SIMULATOR

Every laboratory component (e.g., `SamplingLab.tsx`, `Modulation.tsx`) wraps its engine call inside a `useMemo` dependency array matching the parameter sliders. Moving any slider triggers instant re-renders.

POSSIBLE PROFESSOR FOLLOW-UP

Q: Why not use WebAssembly or Web Workers?
A: For signal vectors under 10,000 points, modern V8 JIT JavaScript engines execute Float64 array loops in under 2 milliseconds, making Web Worker thread context-switching overhead unnecessary.

IMPLEMENTATION STATUS & EVIDENCE

```
Fully implemented (`src/utils/simulations/` and laboratory component files). ---
```

Q2. How do you maintain numeric continuity and prevent plotting artifacts like diagonal lines during instantaneous step transitions?

SIMPLE ANSWER (PRESENTATION / VIVA READY)

> In line coding and digital modulation, digital signals change voltage levels almost instantly. If a graphing library connects two consecutive points across a boundary, it draws an unrealistic sloped diagonal line. To prevent this, our simulator injects dual boundary timestamps separated by a fraction of a microsecond: one immediately before the clock edge with the old amplitude, and one at the edge with the new amplitude. This forces the plotting library to draw crisp vertical transitions.

TECHNICAL EXPLANATION (CODE & ARCHITECTURE)

In `lineCoding.ts` (`encodeLineCode`), each bit duration T_b is divided into points with resolution $dt = T_b/100$. At transition instants (such as $t = nT_b$ or mid-bit $t = nT_b + T_b/2$ in Manchester/RZ), the algorithm pushes:

1. $\{t = t_{\text{edge}} - 0.1\,dt, \text{Amplitude} = V_{\text{prev}}\}$
2. $\{t = t_{\text{edge}}, \text{Amplitude} = V_{\text{next}}\}$

For digital coding staircase signals (DM/ADM), the chart renders with `Line type="stepAfter"`, which holds the reconstructed value across the sample interval.

GOVERNING FORMULA

Boundary formulation:

$$x(t_{\text{edge}}^{-}) = V_{\text{old}}, \quad x(t_{\text{edge}}^{+}) = V_{\text{new}}, \quad \lim_{\Delta t \to 0} [x(t_{\text{edge}}) - x(t_{\text{edge}} - \Delta t)] = \Delta V$$

HOW IT APPEARS IN SIMULATOR

Line coding waveforms show strictly perpendicular vertical clock edges without trapezoidal distortion, and bit boundary dashed markers align with the transitions.

POSSIBLE PROFESSOR FOLLOW-UP

> *Q: Does this introduce duplicate keys in the chart dataset?*
> *A:* No, because the preceding point is offset by $-0.1\,dt$, preserving strict monotonic ordering ($t_k < t_{k+1}$).

IMPLEMENTATION STATUS & EVIDENCE

`Fully implemented (`lineCoding.ts` lines 94–112). --- ### Section B: Sampling & Quantization ---`

SECTION 2

Sampling & Quantization

Nyquist sampling criteria, programmatic aliasing detection, ideal Whittaker–Shannon sinc interpolation, and true MSE measurement.

Q3. What is the Nyquist sampling theorem, and how does your code programmatically evaluate whether aliasing has occurred?

SIMPLE ANSWER (PRESENTATION / VIVA READY)

The Nyquist–Shannon sampling theorem states that a continuous band-limited signal with highest frequency f_m can be reconstructed without distortion if it is sampled at a rate $f_s \geq 2f_m$. In our code, we compute the Nyquist threshold $2f_m$ and compare it directly against the chosen f_s. If $f_s < 2f_m$, the code sets an `isAliased` boolean to true, displays an alert badge, and calculates the apparent alias frequency that folds into the baseband spectrum.

TECHNICAL EXPLANATION (CODE & ARCHITECTURE)

In `sampling.ts` (`simulateSampling`), the parameter `nyquist` is computed as $2f_m$. The condition `isAliased = fs < 2 * fm` drives the UI warning. To determine the apparent alias frequency f_{alias} observed in the time domain, the code determines the closest integer harmonic $n = \text{round}(f_m/f_s)$ and computes the difference $|f_m - nf_s|$.

GOVERNING FORMULA

Nyquist criterion:

$$f_s \geq 2f_m$$

Folded alias frequency in baseband:

$$f_{\text{alias}} = |f_m - nf_s|, \quad \text{where } n = \lfloor \frac{f_m}{f_s} + \frac{1}{2} \rfloor$$

HOW IT APPEARS IN SIMULATOR

In the Sampling laboratory:

- When $f_m = 100$ Hz and $f_s = 500$ Hz, the badge displays green `Valid` with ratio 5.00.
- When f_s is reduced to 150 Hz, the badge turns red `Aliasing Detected` , and the metric table reports `Alias Frequency: 50 Hz` .

POSSIBLE PROFESSOR FOLLOW-UP

Q: What happens if $f_s = 2f_m$ exactly?
A: Mathematically it meets the theoretical minimum, but if samples fall at zero crossings of a sine wave ($\sin(n\pi) = 0$), the signal disappears. In practical systems, an anti-aliasing guard band ($f_s > 2f_m$) is required.

IMPLEMENTATION STATUS & EVIDENCE

`Fully implemented (`sampling.ts` lines 136–154, `SamplingLab.tsx` lines 117–122). ---`

Q4. How does your simulator reconstruct a continuous-time signal from discrete sampled values?

▪ SIMPLE ANSWER (PRESENTATION / VIVA READY)

> Our simulator implements ideal Whittaker–Shannon sinc interpolation. Every quantized sample point acts as a discrete impulse with an amplitude-scaled sinc function centered at its sampling instant. For every continuous point on our timeline, we sum the contributions of all overlapping sinc functions across the window. When the Nyquist criterion is satisfied, the overlapping tails cancel out and recover the continuous sine wave.

▪ TECHNICAL EXPLANATION (CODE & ARCHITECTURE)

In `sampling.ts` (lines 53–66), the array `reconstructed` of size N is generated by a double loop:

```
for(let i = 0; i < numPoints; i++) {
  const t = time[i];
  let sum = 0;
  for(let n = 0; n < samplePoints.length; n++) {
    sum += samplePoints[n].q * sinc(fs * (t - samplePoints[n].t));
  }
  reconstructed[i] = sum;
}
```

where `sinc(x) = Math.sin(Math.PI * x) / (Math.PI * x)` (with `sinc(0) = 1`). Notice that it interpolates over the quantized sample values q_n, accurately demonstrating both sampling and quantization effects.

▪ GOVERNING FORMULA

Whittaker–Shannon interpolation formula:

$$x_r(t) = \sum_{n=0}^{N_s-1} x_q[n]\,\text{sinc}\left(f_s(t-t_n)\right) = \sum_{n=0}^{N_s-1} x_q[n]\frac{\sin\left(\pi f_s(t-n/f_s)\right)}{\pi f_s(t-n/f_s)}$$

▪ HOW IT APPEARS IN SIMULATOR

When checking the "Reconstructed Signal" checkbox, a blue curve renders over the gray original signal. When aliased, the blue curve reconstructs the lower-frequency alias waveform rather than the original signal.

▪ POSSIBLE PROFESSOR FOLLOW-UP

> *Q: Why do the endpoints of the reconstructed wave slightly droop or deviate near $t = 0$ and $t = 50$ ms?*
> *A:* Ideal sinc interpolation assumes an infinite summation from $n = -\infty$ to $+\infty$. Truncating the sum to the finite observation window ($0 \le t \le 50$ ms) causes window truncation / edge boundary errors.

▪ IMPLEMENTATION STATUS & EVIDENCE

`Fully implemented (`sampling.ts` lines 1–2, 53–66). ---`

Q5. How is quantization executed mathematically in your Sampling laboratory, and how are MSE and SNR computed?

SIMPLE ANSWER (PRESENTATION / VIVA READY)

> The signal has normalized boundaries between $-1\ \text{V}$ and $+1\ \text{V}$. We divide this range into discrete intervals using uniform quantization. Each sample amplitude is mapped to the closest discrete level index, clamped to legal bounds, and assigned that level's voltage. The simulator then compares the original continuous signal against the reconstructed signal at every timestamp to compute the true Mean Squared Error and the resulting Signal-to-Noise Ratio in decibels.

TECHNICAL EXPLANATION (CODE & ARCHITECTURE)

In `sampling.ts`, with $V_{\text{min}} = -1\ \text{V}$ and $V_{\text{max}} = +1\ \text{V}$ and $L = 16$ levels:

```
const stepSize = (vMax - vMin) / (levels - 1);
const levelIdx = Math.round((val - vMin) / stepSize);
const clampedIdx = Math.max(0, Math.min(levels - 1, levelIdx));
return vMin + clampedIdx * stepSize;
```

For metrics (lines 69–83), the total squared error is accumulated across all continuous points: $\text{MSE} = \frac{1}{N}\sum_{i=0}^{N-1}(x[i] - x_r[i])^2$, and signal power $P_{\text{sig}} = \frac{1}{N}\sum x[i]^2$. SNR is computed as $10\log_{10}(P_{\text{sig}}/\text{MSE})$.

GOVERNING FORMULA

Uniform quantizer step size:

$$\Delta = \frac{V_{\text{max}} - V_{\text{min}}}{L - 1}$$

Quantization mapping:

$$x_q = V_{\text{min}} + \text{clamp}\left(\left\lfloor \frac{x - V_{\text{min}}}{\Delta} + 0.5 \right\rfloor, 0, L-1\right) \cdot \Delta$$

Signal-to-Noise Ratio:

$$\text{SNR}_{\text{dB}} = 10\log_{10}\left(\frac{P_{\text{signal}}}{\text{MSE}}\right) = 10\log_{10}\left(\frac{\frac{1}{N}\sum_{i=1}^{N} x^2[i]}{\frac{1}{N}\sum_{i=1}^{N}(x[i] - x_r[i])^2}\right)$$

HOW IT APPEARS IN SIMULATOR

The "Performance Measurements" panel displays Quantization Step (≈ 0.1333), Mean Squared Error, Max Absolute Error, and SNR in dB.

POSSIBLE PROFESSOR FOLLOW-UP

> *Q: Why does the code use $(L-1)$ in the denominator instead of L?*
> *A:* When both the minimum $(-1\ \text{V})$ and maximum $(+1\ \text{V})$ boundaries are included as valid representation levels, the number of intervals separating L points is $L-1$. In standard mid-rise quantizers, $\Delta = \frac{2V_p}{L}$, which places boundaries at $\pm(V_p - \Delta/2)$.

IMPLEMENTATION STATUS & EVIDENCE

`Fully implemented (`sampling.ts` lines 30–39, 68–83). --- ### Section C: Digital Baseband Coding (PCM, DPCM, DM, ADM) ---`

SECTION 3

Digital Baseband Coding

PCM bit rates, DPCM predictive feedback, Delta Modulation slope overload detection, and Adaptive Delta Modulation step dynamics.

Q6. How does Pulse Code Modulation (PCM) work in your simulator, and how are bit rate and bandwidth determined?

SIMPLE ANSWER (PRESENTATION / VIVA READY)

PCM performs three sequential steps: sampling at rate f_s, uniform quantization into L discrete levels, and binary encoding where each level is mapped to $n = \lceil \log_2 L \rceil$ bits. Our simulator calculates the transmitted bit rate as $R_b = n \cdot f_s$ bits per second. The reconstructed signal is generated by sinc-interpolating the quantized samples, allowing users to see how increasing L drops the Mean Squared Error and increases SNR.

TECHNICAL EXPLANATION (CODE & ARCHITECTURE)

In `digitalCoding.ts` (`pcmEncode`), the input is sampled at interval $1/f_s$. Each sample is quantized using `quantizeUniform(v, -A, A, levels)`. The bits per sample are computed as:

```
const bitsPerSample = Math.max(1, Math.ceil(Math.log2(levels)));
const bitRate = fs * bitsPerSample;
```

Reconstruction is performed via Whittaker–Shannon interpolation. The metric panel records the resulting trade-off: higher L increases SNR, but proportionally increases the required transmission bit rate R_b.

GOVERNING FORMULA

Bits per sample:

$$n = \lceil \log_2(L) \rceil \quad \text{[bits/sample]}$$

Bit rate:

$$R_b = n \cdot f_s \quad \text{[bps]}$$

Theoretical maximum SNR for a sinusoidal signal in full-scale uniform PCM:

$$\text{SNR}_{\text{ideal}} \approx 6.02n + 1.76 \quad \text{[dB]}$$

HOW IT APPEARS IN SIMULATOR

In the Digital Coding laboratory under the "PCM" tab:

- Selecting $L = 16$ gives `Bits / Sample: 4` and at $f_s = 200$ Hz displays `Bit Rate: 800 bps`.
- Changing L to 64 immediately updates to `Bits / Sample: 6` and `Bit Rate: 1200 bps`, with a corresponding drop in MSE.

POSSIBLE PROFESSOR FOLLOW-UP

Q: What minimum transmission channel bandwidth is needed to transmit this PCM signal without ISI?
A: By Nyquist's criterion for baseband transmission, the minimum theoretical bandwidth is $B_{\min} = \frac{R_b}{2} = \frac{nf_s}{2}$ Hz.

IMPLEMENTATION STATUS & EVIDENCE

`Fully implemented (`digitalCoding.ts` lines 64–108, `DigitalCoding.tsx`). ---`

Q7. How does Differential PCM (DPCM) differ from PCM in your codebase?

SIMPLE ANSWER (PRESENTATION / VIVA READY)

> PCM quantizes and transmits the absolute amplitude of each sample, which wastes bits on redundant information when consecutive samples are correlated. DPCM transmits only the difference between the current sample and a predicted value based on prior samples. In our simulator, the predictor uses the previous reconstructed sample. Because the difference signal has a smaller dynamic range than the raw input, DPCM achieves lower quantization error and higher SNR for the same number of bits.

TECHNICAL EXPLANATION (CODE & ARCHITECTURE)

In `digitalCoding.ts` (`dpcmEncode`), a feedback loop tracks `prevReconstructed` :

```
for(let i = 0; i < numSamples; i++) {
  const v = amplitude * Math.sin(2 * Math.PI * fm * t);
  const diff = v - prevReconstructed;
  const qDiff = quantizeUniform(diff, diffMin, diffMax, levels);
  const q = prevReconstructed + qDiff;
  samplePoints.push({ t, v, pred: prevReconstructed, diff, qDiff, q });
  prevReconstructed = q;
}
```

The receiver reconstructs the signal by accumulating the quantized differences: $x[n] = x[n-1] + e_q[n]$.

GOVERNING FORMULA

Prediction error:

$$e[n] = x[n] - \hat{x}[n]$$

First-order predictor:

$$\hat{x}[n] = \tilde{x}[n-1]$$

Quantized prediction error:

$$e_q[n] = Q(e[n])$$

Reconstructed sample:

$$\tilde{x}[n] = \hat{x}[n] + e_q[n] = \tilde{x}[n-1] + e_q[n]$$

HOW IT APPEARS IN SIMULATOR

Under the "DPCM" tab, toggling the "Predicted Signal" checkbox reveals a dashed purple staircase tracking ahead of the reconstructed curve, showing the prediction feedback loop.

POSSIBLE PROFESSOR FOLLOW-UP

> *Q: What happens if an error occurs during transmission over a channel in DPCM?*
> *A:* Because reconstruction depends on accumulation ($x[n] = x[n-1] + e_q[n]$), an error in one difference sample causes an error that propagates to subsequent samples until reset.

IMPLEMENTATION STATUS & EVIDENCE

`Fully implemented (`digitalCoding.ts` lines 110–160). ---`

Q8. How does Delta Modulation (DM) work, and how does your simulator detect Slope Overload and Granular Noise?

SIMPLE ANSWER (PRESENTATION / VIVA READY)

> Delta Modulation is a 1-bit differential coding scheme that uses only two states: $+1$ (increase by step Δ) or -1 (decrease by step Δ). At each clock edge, it compares the current signal with the previous estimate and outputs 1 bit. Our simulator tests whether the maximum derivative of the input signal exceeds the maximum tracking rate $\Delta \cdot f_s$. If it does, the simulator flags **Slope Overload**. If the step size is too large in flat regions, it flags **Granular Noise**.

TECHNICAL EXPLANATION (CODE & ARCHITECTURE)

In `digitalCoding.ts` (`dmEncode`):

```
const diff = v - prevReconstructed;
const qDiff = diff >= 0 ? delta : -delta;
const q = prevReconstructed + qDiff;
```

Reconstruction uses a Zero-Order Hold (ZOH) staircase. The slope checks (lines 198–205) compare:

1. Max signal derivative: $\max\left|\frac{dx}{dt}\right| = 2\pi f_m A$.
2. DM tracking slope: $S_{\text{DM}} = \Delta \cdot f_s$. If $2\pi f_m A > \Delta \cdot f_s$, `slopeOverload = true`. If $\Delta > 0.1A$, `granularNoise = true`.

GOVERNING FORMULA

Accumulator update:

$$\hat{x}[n] = \hat{x}[n-1] + \Delta \cdot \text{sgn}(x[n] - \hat{x}[n-1])$$

Condition to prevent Slope Overload Distortion:

$$\Delta \cdot f_s \geq \max\left|\frac{dx(t)}{dt}\right| = 2\pi f_m A \implies \Delta \geq \frac{2\pi f_m A}{f_s}$$

HOW IT APPEARS IN SIMULATOR

Under the "DM" tab:

- Setting $f_m = 10\text{ Hz}, A = 1\text{ V}, f_s = 1000\text{ Hz}, \Delta = 0.02$ causes the observation box to show:
  `SLOPE OVERLOAD: The step size (0.02) is too small to follow the fast variations of the input signal.`
- Increasing $\Delta = 0.3$ shows: `GRANULAR NOISE: The step size is unnecessarily large relative to the signal variation...`

POSSIBLE PROFESSOR FOLLOW-UP

> *Q: How does DM compare to PCM in bit rate?*
> *A:* DM transmits only 1 bit per sample ($R_b = f_s$), but requires a much higher sampling rate $f_s \gg 2f_m$ (oversampling) to prevent slope overload.

IMPLEMENTATION STATUS & EVIDENCE

`Fully implemented (`digitalCoding.ts` lines 162–212). ---`

Q9. How does Adaptive Delta Modulation (ADM) resolve the limitations of standard Delta Modulation in your project?

▪ SIMPLE ANSWER (PRESENTATION / VIVA READY)

> Standard DM suffers from a trade-off: small step sizes cause slope overload on steep edges, while large step sizes cause granular noise on flat segments. Adaptive Delta Modulation solves this by adjusting the step size dynamically. When consecutive bits have the same sign (indicating the staircase is falling behind), the step size is scaled up by a factor of 1.5. When bits alternate in sign (indicating hunting around a flat region), the step size is divided by 1.5.

▪ TECHNICAL EXPLANATION (CODE & ARCHITECTURE)

In `digitalCoding.ts` (`admEncode`), the algorithm tracks `prevBit` and updates `currentDelta` :

```
if (bit === prevBit) {
  currentDelta = Math.min(currentDelta * 1.5, maxDelta);
} else {
  currentDelta = Math.max(currentDelta / 1.5, minDelta);
}
```

where $K = 1.5$ is the Jayant adaptation multiplier, $\Delta_{\min} = 0.01$, and $\Delta_{\max} = 1.5A$. The reconstructed waveform tracks steep transients with fewer slope overload artifacts than fixed-step DM.

▪ GOVERNING FORMULA

Jayant step size adaptation rule:

$$\Delta[n] = \begin{cases} \min(\Delta[n-1] \cdot K, \Delta_{\max}), & \text{if } b[n] = b[n-1] \\ \max\left(\frac{\Delta[n-1]}{K}, \Delta_{\min}\right), & \text{if } b[n] \neq b[n-1] \end{cases}$$

where $K = 1.5 > 1$.

▪ HOW IT APPEARS IN SIMULATOR

> Under the "ADM" tab, checking the "Plot Step Size" box overlays an orange curve showing the instantaneous value of $\Delta[n]$. The step size expands along steep slopes and contracts near peaks and zero crossings.

▪ POSSIBLE PROFESSOR FOLLOW-UP

> *Q: What is the Compare tab in the Digital Coding laboratory showing?*
> *A:* It generates an automated comparison table computing Bits/Sample, MSE, SNR, and Bit Rate side-by-side for PCM, DPCM, DM, and ADM on the same input signal.

▪ IMPLEMENTATION STATUS & EVIDENCE

`Fully implemented (`digitalCoding.ts` lines 214–272, `DigitalCoding.tsx` lines 241–281). --- ### Section D: Line Coding & Spectral Analysis ---`

SECTION 4

Line Coding & Spectral Analysis

Binary level-mapping rules, DC content elimination in Manchester and Bipolar AMI, and analytical power spectral density modeling.

Q10. Which line coding schemes are implemented in DigiCommLab, and what are their voltage-level mapping rules?

SIMPLE ANSWER (PRESENTATION / VIVA READY)

DigiCommLab implements six primary baseband line codes: Unipolar NRZ, Polar NRZ, Polar RZ, Manchester, Bipolar AMI, and Differential NRZ. Each scheme maps binary 1s and 0s onto specific voltage levels and transitions. For instance, Manchester introduces a mid-bit transition for every bit, guaranteeing clock recovery, while Bipolar AMI alternates the polarity of 1s to remove the DC component.

TECHNICAL EXPLANATION (CODE & ARCHITECTURE)

In `lineCoding.ts` (`encodeLineCode`), each bit interval T_b is split into two halves (`valFirstHalf` and `valSecondHalf`):

- **Unipolar NRZ**: $1 \to (+A, +A)$, $0 \to (0, 0)$.
- **Polar NRZ**: $1 \to (+A, +A)$, $0 \to (-A, -A)$.
- **Polar RZ**: $1 \to (+A, 0)$, $0 \to (-A, 0)$ (returns to zero at $T_b/2$).
- **Manchester**: $1 \to (+A, -A)$, $0 \to (-A, +A)$ (split-phase).
- **Bipolar AMI**: $0 \to (0, 0)$; $1 \to$ alternates $+A$ and $-A$ via `amiLastPolarity = -amiLastPolarity` .
- **Differential NRZ-M**: $1 \to$ toggles state (`diffCurrentState = -diffCurrentState`); $0 \to$ keeps previous state.

GOVERNING FORMULA

Mathematical level definitions over $t \in [0, T_b)$:

- **Manchester**:

$$p(t) = \begin{cases} +A, & 0 \le t < T_b/2 \\ -A, & T_b/2 \le t < T_b \end{cases} \quad \text{(for bit 1)}, \quad p(t) = \begin{cases} -A, & 0 \le t < T_b/2 \\ +A, & T_b/2 \le t < T_b \end{cases} \quad \text{(for bit 0)}$$

- **Bipolar AMI**:

$$s_n(t) = a_n \cdot \text{rect}\left(\frac{t - nT_b - T_b/2}{T_b}\right), \quad a_n \in \{+A, -A\} \text{ alternately for bit 1}, a_n = 0 \text{ for bit 0}$$

HOW IT APPEARS IN SIMULATOR

Users can enter custom bit sequences (e.g., `101100101`) or click "Randomize". The oscilloscope canvas plots the pulse train with dashed timing grids and bit values labeled above each period.

POSSIBLE PROFESSOR FOLLOW-UP

Q: How does Differential line coding prevent 180-degree phase ambiguity?
A: Because information is encoded in whether a transition occurs rather than the absolute voltage polarity, swapping wires or inverting the channel phase leaves the decoded transitions unchanged.

IMPLEMENTATION STATUS & EVIDENCE

`Fully implemented (`lineCoding.ts` lines 25–143). ---`

Q11. How do you demonstrate that Manchester and Bipolar AMI codes have no DC component?

▪ SIMPLE ANSWER (PRESENTATION / VIVA READY)

A DC component corresponds to non-zero average voltage over time and an impulse at $f = 0$ in the power spectrum. In Manchester coding, every bit period has equal positive and negative durations, so its integral is identically zero for both 1s and 0s. In Bipolar AMI, alternate 1s cancel each other out over time. In our simulator, toggling "Theoretical PSD" shows that the spectral power curves for both Manchester and AMI drop to zero at $f = 0$.

▪ TECHNICAL EXPLANATION (CODE & ARCHITECTURE)

In `lineCoding.ts`:

1. `encodeLineCode` calculates the arithmetic mean `averageValue = sum / sequence.length` across all points.
2. In `calculatePSD`:
   - Manchester PSD formula includes $\sin^2\left(\frac{\pi f T_b}{2}\right)$, which evaluates to zero at $f = 0$.
   - Bipolar AMI PSD formula includes $\sin^2(\pi f T_b)$, which also evaluates to zero at $f = 0$. Both analytical plots confirm $S(0) = 0$.

▪ GOVERNING FORMULA

Manchester Power Spectral Density:

$$S_{\text{Manchester}}(f) = A^2 T_b \,\text{sinc}^2\left(\frac{f T_b}{2}\right) \sin^2\left(\frac{\pi f T_b}{2}\right) \implies \lim_{f \to 0} S_{\text{Manchester}}(f) = 0$$

Bipolar AMI Power Spectral Density:

$$S_{\text{AMI}}(f) = A^2 T_b \,\text{sinc}^2(f T_b) \sin^2(\pi f T_b) \implies \lim_{f \to 0} S_{\text{AMI}}(f) = 0$$

▪ HOW IT APPEARS IN SIMULATOR

When selecting Manchester or Bipolar AMI and toggling the "Show Theoretical PSD" switch, an Area chart displays the spectrum starting at zero power at $f = 0$, unlike Unipolar and Polar NRZ which peak at baseband.

▪ POSSIBLE PROFESSOR FOLLOW-UP

Q: Why is having zero DC content important in telephone and transmission networks?
A: Transmission lines use AC-coupling transformers and DC-blocking capacitors that attenuate DC components; non-zero DC causes baseline wander and introduces detection errors.

▪ IMPLEMENTATION STATUS & EVIDENCE

`Fully implemented (`lineCoding.ts` lines 145–189). ---`

Q12. How is the Power Spectral Density (PSD) calculated in your Line Coding module? Is it an FFT of the input sequence?

SIMPLE ANSWER (PRESENTATION / VIVA READY)

In the Line Coding module, the PSD is calculated using closed-form analytical equations based on the autocorrelation function of each signaling format, rather than an FFT of the active bit sequence. This displays the theoretical ensemble-average spectral profile of an infinitely long, random data stream. It gives students an accurate textbook spectrum for each line code without being distorted by the short length of the custom input string.

TECHNICAL EXPLANATION (CODE & ARCHITECTURE)

In `lineCoding.ts` (`calculatePSD` lines 145–189), the function computes 200 frequency points up to $3R_b$. For each frequency f, it evaluates the theoretical closed-form expressions derived from the Wiener–Khinchin theorem:

- Unipolar NRZ: $P(f) = \frac{A^2 T_b}{4} \operatorname{sinc}^2(fT_b)$
- Polar NRZ: $P(f) = A^2 T_b \operatorname{sinc}^2(fT_b)$
- Polar RZ: $P(f) = \frac{A^2 T_b}{4} \operatorname{sinc}^2\left(\frac{fT_b}{2}\right)$
- Manchester: $P(f) = A^2 T_b \operatorname{sinc}^2\left(\frac{fT_b}{2}\right) \sin^2\left(\frac{\pi f T_b}{2}\right)$
- Bipolar AMI: $P(f) = A^2 T_b \operatorname{sinc}^2(fT_b) \sin^2(\pi f T_b)$

GOVERNING FORMULA

Wiener–Khinchin theorem:

$$S_X(f) = \mathrm{F}\{R_X(\tau)\} = \int_{-\infty}^{\infty} R_X(\tau) e^{-j2\pi f\tau} d\tau = \frac{1}{T_b} |P(f)|^2 \sum_{k=-\infty}^{\infty} R_A[k] e^{-j2\pi k f T_b}$$

where $P(f)$ is the pulse Fourier transform and $R_A[k]$ is the symbol autocorrelation sequence.

HOW IT APPEARS IN SIMULATOR

When clicking "Theoretical PSD", the graph title explicitly states: `Theoretical Power Spectral Density (PSD) — Normalized Frequency Distribution`.

POSSIBLE PROFESSOR FOLLOW-UP

Q: If a student types `00000000` into the box, why does the PSD not change?
A: Because `calculatePSD` plots the theoretical ensemble average for equiprobable independent bits. An empirical periodogram FFT would collapse to a single DC line; keeping the theoretical curve preserves its educational utility.

IMPLEMENTATION STATUS & EVIDENCE

```
Partially implemented (Accurately calculates theoretical formulas; does not perform empirical sample-FFT on the user input). --- ### Section E: Analog & Digital Passband Modulation ---
```

SECTION 5

Analog & Digital Passband Modulation

AM-DSB overmodulation, Hilbert phase-shift SSB, Carson's FM rule, M-QAM Gray mapping, continuous-phase MSK, Box-Muller AWGN, and DFT analysis.

Q13. What mathematical models are used for your analog modulation schemes (AM-DSB, AM-SSB, and WBFM)?

SIMPLE ANSWER (PRESENTATION / VIVA READY)

Our analog engine simulates three schemes:

1. Conventional Double Sideband AM with carrier, generating an envelope that demonstrates over-modulation when the modulation index $\mu > 1$.
2. Single Sideband AM using the phase-shift method, which subtracts the Hilbert-transformed quadrature component to cancel the lower sideband.
3. Wideband FM, which integrates the message frequency into the carrier phase, expanding the transmission bandwidth according to Carson's rule.

TECHNICAL EXPLANATION (CODE & ARCHITECTURE)

In `modulation.ts` (`simulateAnalog`):

- **AM-DSB**: $s(t) = (A_c + m(t))\cos(2\pi f_c t)$ where $A_c = A_m/\mu$. The envelope is computed as $E(t) = |A_c + m(t)|$.
- **AM-SSB (USB)**: The Hilbert transform of $m(t) = A_m \cos(2\pi f_m t)$ is $\hat{m}(t) = A_m \sin(2\pi f_m t)$. The transmitted upper sideband is: $s_{\text{SSB}}(t) = m(t)\cos(2\pi f_c t) - \hat{m}(t)\sin(2\pi f_c t)$.
- **WBFM**: Given modulation index $\beta = \Delta f / f_m$, the phase integral yields: $s_{\text{FM}}(t) = A_c \cos(2\pi f_c t + \beta \sin(2\pi f_m t))$.

GOVERNING FORMULA

AM-DSB with carrier:

$$s_{\text{AM}}(t) = A_c\left[1 + \mu\cos(2\pi f_m t)\right]\cos(2\pi f_c t)$$

Single Sideband (Upper Sideband):

$$s_{\text{USB}}(t) = m(t)\cos(2\pi f_c t) - \hat{m}(t)\sin(2\pi f_c t)$$

Wideband Frequency Modulation:

$$s_{\text{FM}}(t) = A_c \cos\left(2\pi f_c t + 2\pi k_f \int_0^t m(\tau)d\tau\right) = A_c \cos\left(2\pi f_c t + \beta \sin(2\pi f_m t)\right)$$

HOW IT APPEARS IN SIMULATOR

In the Modulation laboratory under "Analog":

- In AM-DSB with $\mu = 1.5$, a red dashed envelope crosses zero, and the observation box warns: `OVER-MODULATION: The modulation index exceeds 1.0. Envelope detection will fail.`
- In WBFM, Carson's rule bandwidth displays dynamically as `Approx Bandwidth (Carson): 2(Δf + fm) Hz`.

POSSIBLE PROFESSOR FOLLOW-UP

Q: What is Carson's rule bandwidth for WBFM?
A: Carson's rule approximates 98% of the power bandwidth as $B_T \approx 2(\Delta f + f_m) = 2f_m(\beta + 1)$.

IMPLEMENTATION STATUS & EVIDENCE

`Fully implemented (`modulation.ts` lines 47–141). ---`

Q14. How are Digital I/Q Modulations (BPSK, QPSK, 8PSK, 16-QAM, 64-QAM) generated and mapped onto carrier waveforms?

SIMPLE ANSWER (PRESENTATION / VIVA READY)

Digital modulation schemes group incoming bits into symbols of length $k = \log_2(M)$. Each symbol is mapped using Gray coding to an In-Phase (I) and Quadrature (Q) coordinate. The transmitted passband waveform is synthesized using quadrature carrier multiplexing: $s(t) = I\cos(2\pi f_c t) - Q\sin(2\pi f_c t)$. This unified I/Q formulation handles phase shift keying and quadrature amplitude modulation under a single architecture.

TECHNICAL EXPLANATION (CODE & ARCHITECTURE)

In `modulation.ts` (`simulateDigital` lines 152–245):

- **BPSK** ($k = 1$): $I = \pm A, Q = 0$.
- **QPSK** ($k = 2$): Gray mapping where $00 \to (-1,-1), 01 \to (-1,1), 11 \to (1,1), 10 \to (1,-1)$, scaled by $0.707A$.
- **8PSK** ($k = 3$): Integer symbol $m \in [0,7]$ maps to phase $\theta_m = \frac{m\pi}{4}$, with $I = A\cos\theta_m, Q = A\sin\theta_m$.
- **16-QAM** ($k = 4$): Split into 2 bits for $I \in \{-3,-1,+1,+3\} \cdot \frac{A}{3}$ and 2 bits for $Q \in \{-3,-1,+1,+3\} \cdot \frac{A}{3}$.
- **64-QAM** ($k = 6$): 3 bits each mapped to $I, Q \in \{-7,-5,-3,-1,+1,+3,+5,+7\} \cdot \frac{A}{7}$. Passband synthesis:

```
modulated[i] = sym.i * Math.cos(2 * Math.PI * fc * t) - sym.q * Math.sin(2 * Math.PI * fc * t);
```

GOVERNING FORMULA

Quadrature Passband Synthesis:

$$s(t) = I(t)\cos(2\pi f_c t) - Q(t)\sin(2\pi f_c t) = \text{Re}\left\{(I(t) + jQ(t))e^{j2\pi f_c t}\right\}$$

General M-QAM constellation coordinates:

$$I_m, Q_m \in \{(2i - 1 - \sqrt{M})d\}, \quad i \in \{1, 2, \ldots, \sqrt{M}\}$$

HOW IT APPEARS IN SIMULATOR

Selecting 16-QAM or 64-QAM displays the square constellation grid under the "Constellation" tab, alongside metrics for `Bits/Symbol: 4` (or `6`), `Bit Rate` , and the synthesized RF time-domain waveform.

POSSIBLE PROFESSOR FOLLOW-UP

Q: Why is Gray coding preferred over natural binary coding in constellation mapping?
A: In Gray coding, adjacent constellation points differ by only 1 bit. When noise pushes a symbol into an adjacent decision region, it results in only a single bit error, minimizing Bit Error Rate (BER).

IMPLEMENTATION STATUS & EVIDENCE

`Fully implemented (`modulation.ts` lines 152-245, 333-338). ---`

Q15. How are Continuous Phase FSK (CPFSK), Minimum Shift Keying (MSK), and GFSK implemented without phase discontinuities?

▪ SIMPLE ANSWER (PRESENTATION / VIVA READY)

Standard Frequency Shift Keying (BFSK) switches abruptly between two separate oscillators, causing phase discontinuities that spread out spectral sidelobes. Continuous Phase FSK (CPFSK) eliminates these discontinuities by integrating the instantaneous frequency continuously over time into a running phase accumulator. MSK is a special case of CPFSK with modulation index $h = 0.5$, which represents the minimum frequency separation for orthogonal coherent detection. GFSK pre-filters the bit stream with a Gaussian filter before integration, creating a compact spectrum suitable for Bluetooth.

▪ TECHNICAL EXPLANATION (CODE & ARCHITECTURE)

In `modulation.ts` (`simulateDigital` lines 291–332):

- **BFSK**: Switches frequencies directly: $s(t) = A\cos(2\pi f_{\text{bit}} t)$, which produces phase jumps.
- **CPFSK / MSK**: Accumulates phase:

```
const h = scheme === 'MSK' ? 0.5 : 1.0;
const deltaF = h * symbolRate / 2;
const freq = fc + (currentBit === '1' ? deltaF : -deltaF);
currentPhase += 2 * Math.PI * freq * (1/fs);
modulated[i] = amplitude * Math.cos(currentPhase);
```

- **GFSK**: Smooths the discrete bit transitions by convolving them with a Gaussian window $w[k] = \exp\left(-\frac{k^2}{2\sigma^2}\right)$ across a symbol duration before updating the phase accumulator.

▪ GOVERNING FORMULA

Continuous phase integration:

$$s(t) = A_c \cos(\theta(t)), \quad \theta(t) = 2\pi \int_0^t f(\tau)d\tau = 2\pi f_c t + 2\pi h \sum_k a_k \int_{-\infty}^t g(\tau - kT_s)d\tau$$

MSK modulation index condition:

$$h = \Delta f \cdot T_s = |f_1 - f_0| \cdot T_b = 0.5$$

▪ HOW IT APPEARS IN SIMULATOR

Selecting MSK and viewing the "Spectrum" tab shows that spectral sidelobes decay significantly faster than in standard BASK or BFSK, illustrating the benefits of continuous phase tracking.

▪ POSSIBLE PROFESSOR FOLLOW-UP

Q: What are the constellation points for MSK?
A: Because MSK maintains continuous phase with constant envelope, its constellation trajectory is a continuous circle of radius A in the I/Q plane rather than static discrete clusters.

▪ IMPLEMENTATION STATUS & EVIDENCE

`Fully implemented (`modulation.ts` lines 291–332). ---`

Q16. How is the Additive White Gaussian Noise (AWGN) channel modeled, and how does your simulator generate Gaussian random variables in JavaScript?

▪ SIMPLE ANSWER (PRESENTATION / VIVA READY)

> Standard JavaScript `Math.random()` generates uniform random numbers between 0 and 1, not Gaussian noise. To simulate a physical AWGN channel, our code implements the **Box–Muller transformation**, which converts two independent uniform random variables into standard normal Gaussian samples. We scale this noise according to the user's selected SNR in dB and add it to both the time-domain RF waveform and the I/Q constellation points.

▪ TECHNICAL EXPLANATION (CODE & ARCHITECTURE)

In `modulation.ts` (`generateAWGN` lines 7–12):

```
export function generateAWGN(stdDev: number): number {
  let u1 = 0, u2 = 0;
  while (u1 === 0) u1 = Math.random();
  while (u2 === 0) u2 = Math.random();
  return stdDev * Math.sqrt(-2.0 * Math.log(u1)) * Math.cos(2.0 * Math.PI * u2);
}
```

The noise standard deviation σ is derived from the user's slider `snrDb`:

```
const noiseStdDev = Math.pow(10, -snrDb / 20) * (amplitude / Math.sqrt(2));
```

This noise is added to `sym.i`, `sym.q`, and the time-domain samples `modulated[i] += generateAWGN(noiseStdDev)`.

▪ GOVERNING FORMULA

Box–Muller transform:

$$Z = \sigma\sqrt{-2\ln(U_1)}\cos(2\pi U_2), \quad U_1, U_2 \sim \mathrm{U}(0,1) \implies Z \sim \mathrm{N}(0,\sigma^2)$$

Noise standard deviation from SNR (assuming carrier power $P_s = A^2/2$):

$$\mathrm{SNR} = \frac{P_s}{\sigma^2} \implies \sigma = \frac{A}{\sqrt{2}} \cdot 10^{-\frac{\mathrm{SNR}_{\mathrm{dB}}}{20}}$$

▪ HOW IT APPEARS IN SIMULATOR

In the Modulation laboratory under the "Constellation" tab:

- Setting SNR to 35 dB (Clean) produces tightly focused constellation points.
- Lowering SNR to 8 dB (Noisy) scatters the I/Q points into wide Gaussian clouds, visibly demonstrating how noise causes symbol decision errors.

▪ POSSIBLE PROFESSOR FOLLOW-UP

> *Q: Why does the Box-Muller while loop check `while(u1 === 0)`?*
> *A:* $\ln(0)$ evaluates to $-\infty$, which would produce `NaN` values and break the simulation array.

▪ IMPLEMENTATION STATUS & EVIDENCE

`Fully implemented (`modulation.ts` lines 6–12, 181–184, 264–276). ---`

Q17. How is the frequency spectrum calculated in the Modulation laboratory?

SIMPLE ANSWER (PRESENTATION / VIVA READY)

> The frequency spectrum is calculated using a Discrete Fourier Transform (DFT) implemented directly in TypeScript. It computes the magnitude of the signal across 200 discrete frequency bins from 0 Hz up to several times the carrier frequency. This converts the time-domain modulated signal into an empirical frequency spectrum, allowing users to see the carrier spike, sidebands, and spectral roll-off.

TECHNICAL EXPLANATION (CODE & ARCHITECTURE)

In `modulation.ts` (`calculateMagnitudeSpectrum` lines 15–37), the signal of length N is analyzed across `numBins = 200`:

```
for (let k = 0; k < numBins; k++) {
  const f = k * freqStep;
  let re = 0, im = 0;
  for (let n = 0; n < N; n += step) {
    const t = n / fs;
    const angle = 2 * Math.PI * f * t;
    re += signal[n] * Math.cos(angle);
    im -= signal[n] * Math.sin(angle);
  }
  const mag = Math.sqrt(re * re + im * im) / (N / step);
  spectrum.push({ f, magnitude: mag });
}
```

This DFT implementation operates on downsampled points to maintain 60 FPS slider interactions in the browser.

GOVERNING FORMULA

Discrete Fourier Transform magnitude evaluation:

$$X(f_k) = \frac{1}{N'} \sum_{n=0}^{N'-1} s(t_n) e^{-j 2\pi f_k t_n} = \frac{1}{N'} \sum_{n=0}^{N'-1} s(t_n) \left[\cos(2\pi f_k t_n) - j \sin(2\pi f_k t_n)\right]$$

$$|X(f_k)| = \frac{1}{N'} \sqrt{\left(\sum s(t_n) \cos(2\pi f_k t_n)\right)^2 + \left(\sum s(t_n) \sin(2\pi f_k t_n)\right)^2}$$

HOW IT APPEARS IN SIMULATOR

Under the "Spectrum" tab:

- In AM-DSB, the plot displays three distinct spikes: carrier f_c, lower sideband $f_c - f_m$, and upper sideband $f_c + f_m$.
- In AM-SSB, the lower sideband disappears, leaving only $f_c + f_m$.

POSSIBLE PROFESSOR FOLLOW-UP

> *Q: Why did you implement a direct DFT rather than an FFT algorithm like Cooley–Tukey?*
> *A:* An FFT requires array lengths of 2^p $(2048, 4096)$ and computes all bins up to $f_s/2$. A direct DFT allows calculating exactly 200 linearly spaced bins focused on the region of interest around f_c without window padding, keeping runtime around 3–5 ms.

IMPLEMENTATION STATUS & EVIDENCE

`Fully implemented (`modulation.ts` lines 14–37). --- ### Section F: Validation, Testing & System Integration ---`

SECTION 6

Validation and Implementation Limitations

Numerical validation against theoretical benchmarks, engineering approximations, and honest answers to challenging professor viva questions.

Q18. How did you verify the numerical accuracy of your simulations against established theoretical benchmarks?

SIMPLE ANSWER (PRESENTATION / VIVA READY)

We validated our algorithms against textbook benchmarks across three criteria:

1. Sinc interpolation MSE drops toward zero when $f_s \geq 2f_m$ and increases when $f_s < 2f_m$.
2. For line codes, we verified that analytical formulas match standard textbook spectral nulls (such as the DC null in Manchester and AMI).
3. In analog modulation, we verified that AM-DSB sidebands appear at $f_c \pm f_m$, and that Carson's bandwidth formula matches the measured FM spectrum spread.

TECHNICAL EXPLANATION (CODE & ARCHITECTURE)

During verification:

- For uniform quantization, the measured SNR approximates the theoretical formula $\text{SNR} \approx 6.02n + 1.76\,\text{dB}$.
- For Delta Modulation, slope overload was validated by setting parameters above and below $2\pi f_m A = \Delta \cdot f_s$.
- The Box-Muller Gaussian noise generator was validated by checking that the constellation variance matches $\sigma^2 = \frac{A^2}{2} 10^{-\text{SNR}_{\text{dB}}/10}$.

GOVERNING FORMULA

PCM SNR benchmark comparison:

$$\text{SNR}_{\text{theoretical}} \approx 6.02n + 1.76\ \text{dB} \quad \text{vs.} \quad \text{SNR}_{\text{measured}} = 10\log_{10}\left(\frac{\sum x^2[i]}{\sum (x[i] - x_r[i])^2}\right)$$

HOW IT APPEARS IN SIMULATOR

The "Analysis" tabs in Digital Coding and Modulation display performance metrics (MSE, SNR, Bit Rate, Bits/Symbol) that update dynamically as parameters change.

POSSIBLE PROFESSOR FOLLOW-UP

Q: Does the PCM simulation SNR match $6.02n + 1.76$ *dB exactly?*
A: It closely matches but deviates slightly because our simulator measures MSE against the reconstructed continuous sinc waveform over a finite window, which includes edge truncation effects, rather than evaluating discrete sample points alone.

IMPLEMENTATION STATUS & EVIDENCE

```
Fully implemented (All simulation helper files). ---
```

Q19. What approximations or engineering trade-offs were made to ensure the simulator runs smoothly in a web browser?

SIMPLE ANSWER (PRESENTATION / VIVA READY)

To keep the application responsive and interactive without lagging, three primary approximations were made:

1. Display datasets are downsampled to roughly 400–600 points so Recharts can render responsive SVG vectors at 60 FPS.
2. In ideal sinc reconstruction, the summation is truncated to the 50 ms simulation window rather than an infinite time horizon.
3. Pulse shaping for digital I/Q modulations uses rectangular pulses rather than Root-Raised Cosine (RRC) filtering, simplifying the visualization of basic phase transitions.

TECHNICAL EXPLANATION (CODE & ARCHITECTURE)

1. **Downsampling**: `plotFs = \max(10000, 50 f_m)` computes signals with microsecond resolution. Display arrays are downsampled via `step = Math.floor(N / displayPoints)`, while preserving key transition and sample coordinates.
2. **Windowing**: Sinc reconstruction sums over $N_s \approx 25$ to 125 sample points rather than $\pm\infty$.
3. **Rectangular Pulses**: $I(t)$ and $Q(t)$ are held constant over each symbol period T_s, resulting in $\text{sinc}^2(fT_s)$ spectral sidelobes rather than the zero-ISI response of raised-cosine filtering.

GOVERNING FORMULA

Rectangular pulse shaping:

$$p(t) = \text{rect}\left(\frac{t - T_s/2}{T_s}\right) \iff P(f) = T_s \,\text{sinc}(fT_s)e^{-j\pi fT_s}$$

HOW IT APPEARS IN SIMULATOR

The spectrum of BPSK and QPSK exhibits visible sinc^2 sidelobes around f_c, accurately reflecting the frequency characteristics of rectangular baseband pulses.

POSSIBLE PROFESSOR FOLLOW-UP

Q: How could you eliminate those sidelobes in practice?
A: By convolving symbols with a Root-Raised Cosine (RRC) pulse-shaping filter with roll-off factor $\alpha \in [0.2, 0.5]$, which bandlimits the spectrum to $B = \frac{R_s}{2}(1 + \alpha)$.

IMPLEMENTATION STATUS & EVIDENCE

`Fully implemented (Architectural design choice documented across codebase). ---`

Q20. How does DigiCommLab handle state updates and maintain synchronization across its four laboratory modules?

▪ SIMPLE ANSWER (PRESENTATION / VIVA READY)

> DigiCommLab is built as a modular single-page application where each laboratory component manages its own isolated state via React hooks. The application uses client-side routing at the root URL `/`, using smooth coordinate-based scrolling rather than URL hash fragments to move between modules. Shared styling tokens, typography, and SVG trace backgrounds are centralized in `globals.css` and `layout.tsx`, ensuring visual and functional consistency across all experiments.

▪ TECHNICAL EXPLANATION (CODE & ARCHITECTURE)

In `src/app/page.tsx`, the layout arranges the four components sequentially: `<SamplingLab />`, `<DigitalCoding />`, `<LineCoding />`, and `<Modulation />`. Section scrolling is handled by `scrollToSection` in `src/utils/scroll.ts`:

```
const elementPosition = element.getBoundingClientRect().top + window.scrollY;
window.scrollTo({ top: elementPosition - 80, behavior: 'smooth' });
```

This factors in the 80 px fixed navbar height, keeping the URL at `/` without causing hash redirects.

▪ GOVERNING FORMULA

Scroll offset compensation:

$$Y_{\text{target}} = Y_{\text{element}} - H_{\text{navbar}} = (y_{\text{rect}} + y_{\text{scroll}}) - 80 \text{ px}$$

▪ HOW IT APPEARS IN SIMULATOR

Clicking "Experiments" in the navbar or selecting any card in the Hero 3D carousel scrolls the viewport directly to that module.

▪ POSSIBLE PROFESSOR FOLLOW-UP

> *Q: Are simulations running in the background when their section is scrolled off-screen?*
> *A:* No; React re-evaluates `useMemo` blocks only when input slider states change, so idle modules consume zero CPU cycles while viewing other experiments.

▪ IMPLEMENTATION STATUS & EVIDENCE

`Fully implemented (`src/app/page.tsx`, `src/utils/scroll.ts`, `ModuleCarousel.tsx`). ---`

Viva Defense: Answers to Potential Implementation Challenges

Professors frequently probe edge cases and discrepancies between pure textbook theory and real software implementations. Use these honest, technically rigorous responses during evaluation:

Challenge 1: "In your Line Coding module, if I enter `00000000`, why does the PSD not change to a DC impulse?"

Underlying Technical Reality: `calculatePSD()` in `lineCoding.ts` evaluates closed-form theoretical formulas for random, equiprobable bits. It does not compute an empirical FFT of the user's string.

> ***Recommended Viva Response:*** > *"That is an intentional architectural trade-off. The Line Coding module plots the theoretical ensemble-average power spectral density derived via the Wiener–Khinchin theorem for long, equiprobable random bit streams. An empirical periodogram of an 8-bit input would be distorted by windowing artifacts and would not reflect the true statistical spectrum of the line code. We chose to display the textbook theoretical distribution to support learning, though computing a real-time periodogram FFT is a natural next step for the tool."* ---

Challenge 2: "Why does your Sinc reconstructed wave deviate from the original sine wave near $t = 0$ and $t = 50\text{ ms}$?"

Underlying Technical Reality: Ideal Whittaker–Shannon interpolation requires an infinite sum $\sum_{n=-\infty}^{+\infty}$. The code sums over only the 50 ms window ($N_s \approx 25$ to 125 samples).

Recommended Viva Response: > *"That deviation is a window truncation effect. Shannon's interpolation theorem assumes an infinite series of sinc functions. Because we truncate the simulation to a 50 ms window, samples outside that range are omitted, causing the reconstructed sum to droop near the boundaries. In practical hardware, this issue is addressed using causal lowpass reconstruction filters with finite impulse responses."* ---

Challenge 3: "In your Sampling module, why does the uniform quantizer divide by $(L - 1)$ instead of L?"

Underlying Technical Reality: In `sampling.ts`, `stepSize = (vMax - vMin) / (levels - 1)`. Textbooks typically define a uniform quantizer step size as $\Delta = \frac{2 V_p}{L}$.

Recommended Viva Response: > *"Our implementation includes both extreme endpoints (-1.0 V and $+1.0\text{ V}$) as explicit representation levels. For L discrete levels that span both endpoints, there are $L-1$ intermediate step intervals. In standard commercial mid-rise quantizers, the step is $\Delta = \frac{2 V_p}{L}$, which places representation levels at $\pm \frac{\Delta}{2}, \pm \frac{3\Delta}{2}$. Both approaches are valid depending on whether the endpoints or intervals are constrained, though the mid-rise convention is more common in commercial hardware."* ---

Challenge 4: "In digital modulation, your RF waveform has abrupt phase jumps. Are you using pulse shaping?"

Underlying Technical Reality: Rectangular pulse shaping is used for BPSK, QPSK, and QAM. Root-Raised Cosine (RRC) filtering is not implemented (except for a Gaussian approximation in GFSK).

Recommended Viva Response: > *"In the current implementation, we use rectangular baseband pulses for clarity in visualizing raw symbol transitions and constellation points. This results in $\operatorname{sinc}^2(f T_s)$ spectral sidelobes around the carrier. For GFSK, we implemented Gaussian pre-filtering to show how smoothing reduces spectral leakage. Adding an adjustable Root-Raised Cosine (RRC) filter with variable roll-off α is a logical next enhancement."* ---

Challenge 5: "Why is Quantization Levels no longer an adjustable slider in your Sampling laboratory?"

Underlying Technical Reality: The quantization levels button group was removed from `SamplingLab.tsx` in a previous refinement to simplify the control panel, leaving quantization fixed at $L = 16$.

Recommended Viva Response: > *"In the Sampling laboratory, the focus is placed on the sampling theorem, Nyquist rate verification, and aliasing frequency detection, so quantization is fixed at $L=16$ (4-bit resolution). To explore the effects of changing quantization levels ($L = 4$ through 256) on MSE, bit rate, and SNR, we provide the dedicated Digital Coding laboratory directly below it."* ---

SECTION 7

Code-to-Theory Mapping Master Table

Complete architectural reference connecting every visual experiment and UI output directly to its underlying TypeScript engine and mathematical formula.

FEATURE / CONCEPT	ACTUAL SOURCE FILE & FUNCTION	MATHEMATICAL METHOD / ALGORITHM	VISIBLE OUTPUT IN SIMULATOR	IMPLEMENTATION STATUS
Continuous Signal Generation	`sampling.ts` & `digitalCoding.ts` (`generateInputSignal`)	Analytical sinusoidal evaluation: $x(t) = A\sin(2\pi f_m t)$ at oversampled grid $f_{\text{sim}} = \max(10000, 50f_m)$	Smooth baseline curve on time-domain canvas	**Fully implemented**
Nyquist Sampling & Aliasing	`sampling.ts` (`simulateSampling`)	Discrete sampling at $t_n = n/f_s$; condition $f_s < 2f_m$; apparent alias $f_{\text{alias}} = \lvert f_m - \text{round}(f_m/f_s)f_s \rvert$	Red "Aliasing Detected" badge, numerical f_{alias}, discrete red points	**Fully implemented**
Whittaker–Shannon Sinc Reconstruction	`sampling.ts` (`simulateSampling`)	Truncated Whittaker–Shannon interpolation: $x_r(t) = \sum_n q_n \,\text{sinc}(f_s(t - t_n))$	Blue continuous reconstructed waveform overlaid on original signal	**Fully implemented**
Uniform Quantization (Sampling Lab)	`sampling.ts` (`simulateSampling`)	Linear quantizer with step $\Delta = \frac{V_{\max} - V_{\min}}{L-1}$ for $L = 16$ fixed levels	Green quantized points, MSE, Max Error, SNR metrics	**Fully implemented**
Pulse Code Modulation (PCM)	`digitalCoding.ts` (`pcmEncode`)	Absolute sample uniform quantization with $L \in \{4, \dots, 256\}$, $n = \lceil \log_2 L \rceil$, $R_b = nf_s$	Reconstructed curve, discrete quantized dots, SNR, bit rate	**Fully implemented**
Differential PCM (DPCM)	`digitalCoding.ts` (`dpcmEncode`)	1st-order previous-sample prediction:	Purple predicted staircase, green reconstructed waveform, difference metrics	**Fully implemented**
Delta Modulation (DM)	`digitalCoding.ts` (`dmEncode`)	1-bit differential tracking:	Green staircase waveform, "Slope Overload" / "Granular Noise" flags	**Fully implemented**
Adaptive Delta Modulation (ADM)	`digitalCoding.ts` (`admEncode`)	Jayant-multiplier adaptation: $\Delta[n] = \Delta[n-1] \times 1.5$ (same sign) or $\Delta[n-1]/1.5$ (sign flip)	Dynamic staircase tracking, togglable orange step-size trajectory	**Fully implemented**

FEATURE / CONCEPT	ACTUAL SOURCE FILE & FUNCTION	MATHEMATICAL METHOD / ALGORITHM	VISIBLE OUTPUT IN SIMULATOR	IMPLEMENTATION STATUS
Line Code Waveform Generation	`lineCoding.ts` (`encodeLineCode`)	Deterministic level mapping with dual points per bit/transition boundary for clean vertical edges	Oscilloscope-style square waveform with vertical bit partition lines	**Fully implemented**
Line Code Spectral Density (PSD)	`lineCoding.ts` (`calculatePSD`)	Closed-form theoretical autocorrelation Fourier transforms (e.g., sinc^2, Manchester null at DC)	Area chart of normalized power distribution vs. frequency	**Partially implemented** (Analytical formula, not sequence FFT)
Analog Modulation (AM-DSB, AM-SSB, WBFM)	`modulation.ts` (`simulateAnalog`)	AM: $(A_c + m(t))\cos(\omega_c t)$; SSB: $m(t)\cos(\omega_c t) - \hat{m}(t)\sin(\omega_c t)$; FM: $A_c \cos(\omega_c t + \beta \sin(\omega_m t))$	High-frequency modulated carrier, AM envelope tracker, Carson bandwidth	**Fully implemented**
Digital I/Q Modulations (BPSK, QPSK, 8PSK, QAM)	`modulation.ts` (`simulateDigital`)	Gray-coded constellation symbol lookup; passband synthesis: $s(t) = I\cos(\omega_c t) - Q\sin(\omega_c t)$	Passband carrier, square data stream, I/Q scatter diagram	**Fully implemented**
Digital Continuous Phase FSK (CPFSK, MSK, GFSK)	`modulation.ts` (`simulateDigital`)	Continuous phase integration $\theta(t) = \int 2\pi f(\tau)d\tau$; MSK modulation index $h = 0.5$; GFSK Gaussian smoothing	Continuous phase RF waveform, spectrum with suppressed sidelobes	**Fully implemented**
AWGN Channel Emulation	`modulation.ts` (`generateAWGN`)	Box–Muller transformation: $\sigma = 10^{-\text{SNR}_{\text{dB}}/20}$. 2		
c-2.7,0,-7.17,-2.7,-13.5,-8c-5.8,-5.3,-9.5,-10,-9.5,-14				
c0,-2,0.3,-3.3,1,-4c1.3,-2.7,23.83,-20.7,67.5,-54				
c44.2,-33.3,65.8,-50.3,66.5,-51c1.3,-1.3,3,-2,5,-2c4.7,0,8.7,3.3,12,10				
s173,378,173,378c0.7,0,35.3,-71,104,-213c68.7,-142,137.5,-285,206.5,-429				
c69,-144,104.5,-217.7,106.5,-221				
l0 -0				
c5.3,-9.3,12,-14,20,-14				
H400000v40H845.2724				
s-225.272,467,-225.272,467s-235,486,-235,486c-2.7,4.7,-9,7,-19,7				
c-6,0,-10,-1,-12,-3s-194,-422,-194,-422s-65,47,-65,47z				
M834 80h400000v40h-400000z"/>A injected into I/Q and time domain	Scattering of constellation clusters and noisy RF carrier	**Fully implemented**		

FEATURE / CONCEPT	ACTUAL SOURCE FILE & FUNCTION	MATHEMATICAL METHOD / ALGORITHM	VISIBLE OUTPUT IN SIMULATOR	IMPLEMENTATION STATUS
DFT Spectrum Estimation	`modulation.ts` (`calculateMagnitudeSpectrum`)	Direct time-domain Discrete Fourier Transform summation over N bins: $\|X(f)\| = \frac{1}{N}\|\sum s[n]e^{-j2\pi f t_n}\|$	Frequency spectrum area chart showing carrier spikes and sidebands	**Fully implemented**

SECTION 8

Final Viva Revision Sheet

Concise summary of essential formulas, foundational concepts, and presentation pitfalls to review before presenting.

Part A: 15 Core Formulas to Memorize

1. **Nyquist Criterion:** $f_s \geq 2f_m$
2. **Aliasing Foldover Frequency:** $f_{\text{alias}} = |f_m - nf_s|$, where $n = \text{round}(f_m/f_s)$
3. **Whittaker–Shannon Interpolation:** $x_r(t) = \sum_n x_q[n]\,\text{sinc}(f_s(t - n/f_s))$
4. **Normalized Sinc Function:** $\text{sinc}(u) = \frac{\sin(\pi u)}{\pi u}$
5. **PCM Uniform Step Size:** $\Delta = \frac{V_{\max} - V_{\min}}{L-1}$
6. **PCM Transmission Bit Rate:** $R_b = n \cdot f_s = \lceil \log_2 L \rceil \cdot f_s$
7. **PCM Ideal Quantization SNR:** $\text{SNR}_{\text{dB}} \approx 6.02n + 1.76$ dB
8. **DPCM Prediction Error:** $e[n] = x[n] - \hat{x}[n]$, with